

AWS Deployment Guide

Taking the PilotFish FHIR R4 Expandable Platform
from local Docker Compose to a real AWS environment

Implementation-oriented · CapabilityStatement 0.6.0 · 2026-08-05

Audience. Platform engineers, DevOps, and FHIR integration leads who would open a ticket titled “stand up the FHIR R4 PilotFish demo in AWS.” This is a deployability map — not an automatic production certification. Keep the companion *Expert Due Diligence* PDF in scope for FHIR feature gaps.

Recommended compute path for first AWS env: ECS Fargate (EIP + Web UI) + RDS for SQL Server + Application Load Balancer (ACM TLS) + Secrets Manager + EFS (shared bulk-export / file dirs) + Keycloak on ECS *or* Amazon Cognito as token issuer. EC2 single-host Docker Compose is acceptable for a sandboxed staging box only.

1. What you are actually deploying

The local demo is four containers (+ one SQL init job). Your AWS design must preserve those roles, even if services move to managed AWS equivalents.

Demo service	Role	Local port	AWS-managed destination
pilotfish	eiPlatform Tomcat + custom FHIR JARs (JWT, HAPI validate, Bulk)	8110 → 8080	ECS Fargate service (or EC2) behind internal/public ALB
sqlserver	FhirR4PlatformDemo DB (resources, search, export jobs)	14338 → 1433	Amazon RDS for SQL Server (Multi-AZ for prod)
keycloak	OIDC issuer + JWKS for Bearer on writes/\$export	8112 → 8080	Keycloak on ECS/EC2 or Cognito User Pool / Enterprise IdP
webui	Demo operator UI (optional in prod)	8111	ECS task on private ALB, or omit and use API-only
host volumes output/, input/, logs/	Bulk NDJSON, outbound queue, Tomcat logs	bind mounts	EFS mount + CloudWatch Logs (+ S3 lifecycle for exports)

Critical filesystem contract inside the EIP container:

```
/opt/pilotfish/output/fhir-store
/opt/pilotfish/output/outbound-responses
/opt/pilotfish/output/bulk-export/{jobId}/
/opt/pilotfish/input/outbound
/usr/local/tomcat/webapps/eip/eip-root/ # routes + environment-settings.conf
```

PilotFish license reality: the demo image is built FROM `pilotfish-eip:23R1`. AWS deployments need a licensed eiPlatform runtime and an agreed image distribution path (customer registry vs PilotFish-provided base). Do not assume the sandbox demo image is redistribution-cleared for production.

2. Reference architecture (first real AWS environment)

Network

- **VPC** with 2+ AZs; public subnets (ALB) + private app subnets (ECS) + private data subnets (RDS).
- **ALB** internet-facing (or internal-only for VPN/PrivateLink partners) with ACM certificate.
- Optional **AWS WAF** on ALB for FHIR API abuse controls / IP allowlists.
- No public security-group ingress to RDS, Keycloak admin, or SQL ports.
- Nat Gateway(s) so private tasks can pull from ECR and call JWKS / outbound FHIR.

Logical request path

```
Client
→ ALB :443 path /eip/*           → Target group: EIP tasks :8080
→ ALB :443 path /auth/*         → Target group: Keycloak (if self-hosted)
→ ALB :443 path / (ops UI)      → Target group: Web UI :8111 [optional]
EIP task
→ RDS SQL Server :1433 (SG-locked)
→ EFS /opt/pilotfish/output & /input
→ OIDC JWKS (Keycloak SG or Cognito public HTTPS)
Bulk client
→ kickoff /$export → poll status → GET /$export-file (same ALB host)
```

Sizing starting points (non-prod)

Component	Starter size	Notes
EIP task	2 vCPU / 4 GB (Fargate)	Raise heap via CATALINA_OPTS; Bulk jobs are memory-/IO-heavy
Web UI task	0.25 vCPU / 0.5 GB	Omit for API-only prod

Component	Starter size	Notes
Keycloak task	1 vCPU / 2 GB + RDS/Aurora Postgres	Never use start-dev in AWS
RDS SQL Server	db.r6i.large Multi-AZ (prod)	db.t3.medium OK for demo/staging only
EFS	General Purpose, bursting→elastic	Mandatory if EIP task count > 1 for Bulk files

3. Compute & auth decision matrix

Choice	Use when	Avoid when
ECS Fargate + RDS + ALB	You want AWS-native ops, autoscale later, clean SG boundaries	Team cannot containerize licensed EIP base image yet
EC2 + Docker Compose (lift)	48-hour private staging; learning env; migration workshop	Anything customer-facing or HA; demo passwords persist
EKS	Org standard is Kubernetes + GitOps already	First FHIR stand-up — overhead usually not worth it yet
Self-hosted Keycloak	You need realm parity with demo / custom claims soon	You lack Keycloak ops skill — prefer Cognito/Okta/Entra
Amazon Cognito (or Entra/Okta)	Enterprise IdP already exists; JWT validation only needs issuer/JWKS	You must stay byte-identical to demo realm without mapping work

Issuer / JWKS gotcha (this demo): environment-settings.conf currently has `fhir.oauth.jwks=http://keycloak:8080/...` and `fhir.oauth.issuer=http://localhost:8112/...`. In AWS you must set a single public issuer URL that matches access-token iss, and a JWKS URL the EIP task can reach. Mismatched issuer is the #1 Bearer 401 cause after lift.

4. Phased implementation tickets

Phase A — Foundation (ticket-sized)

ID	Work item	Done when
A1	VPC + subnets + NAT + VPC Flow Logs	2 AZ skeleton exists in account/region
A2	ACM cert for fhir.example.com (+ auth host if split)	ACM Issued; DNS validated
A3	ECR repos for eip + webui (+ keycloak if used)	Push IAM for CI role works
A4	Secrets Manager secrets (see §6)	No plaintext passwords in task defs
A5	RDS SQL Server + security group + parameter group	Private endpoint; encrypted at rest
A6	Run sql/*.sql migration job (ECS one-shot or SSM)	FhirR4PlatformDemo schema Phase 1–6 present

Phase B — Runtime

ID	Work item	Done when
B1	Build EIP image FROM licensed base + custom modules	Image in ECR; SBOM/scan attached
B2	EFS + access point; mount output/input paths	Write test file visible across tasks
B3	ECS service EIP (desired count 1 initially)	GET /eip/rest/fhir/metadata → 200 CapStatement
B4	ALB listener rules + target health	Public HTTPS FHIR base URL works
B5	Wire PublicFhirBase + CapStatement url to ALB HTTPS host	export Content-Location uses public host
B6	IdP: Keycloak prod mode or Cognito app client	JWKS fetch OK from EIP task; Bearer write succeeds
B7	Optional Web UI task (private)	Ops can invoke without localhost tunnels

Phase C — Hardening & ops

ID	Work item	Done when
C1	CloudWatch Logs + metrics + alarms (5xx, CPU, RDS)	Pager/slack on ALB 5xx + RDS storage
C2	Close anonymous reads if required; enforce Bearer broadly	Policy matches CapStatement & security model
C3	Backup: RDS snapshots + EFS backup / S3 export copy	Restore drill documented

ID	Work item	Done when
C4	WAF / private connectivity for B2B clients	Threat model signed by security
C5	Run living test plan against AWS base URL	documents/test-results.html green from CI
C6	Multi-AZ RDS + EIP desiredCount $\geq$ 2 only after EFS proven	Bulk file download works mid-failover test

5. Security group matrix

SG	Ingress	Egress
sg-alb	443 from Internet or corp CIDR / PrivateLink	to sg-eip:8080, sg-webui:8111, sg-kc:8080
sg-eip	8080 from sg-alb only	1433→sg-rds; 443→JWKS/ECR/S3; NFS→EFS; DNS
sg-webui	8111 from sg-alb (or VPN CIDR)	1433→sg-rds; 8080→sg-eip; 8080→sg-kc; HTTPS out
sg-kc	8080 from sg-alb + sg-eip (+ admin from break-glass CIDR)	5432→KC DB if used; HTTPS out
sg-rds	1433 from sg-eip + sg-webui (+ migrate task SG)	None required
sg-efs	2049 from sg-eip (+ webui if it mounts)	—

6. Secrets Manager & configuration mapping

Replace every demo credential. Prefer injecting Secrets Manager ARNs into the ECS task definition rather baking values into environment-settings.conf in the image.

6.1 Secrets to create

Secret name (example)	Keys	Used by
fhir/rds/app	username, password, host, port, dbname, jdbc_url	EIP environment-settings / task env; Web UI DB_*
fhir/oauth/client	client_id, client_secret	Backend services / Web UI token fetch
fhir/oauth/issuer	issuer_url, jwks_url	EIP JWT processor + CapStatement security
fhir/keycloak/admin	admin_user, admin_password	Keycloak bootstrap only (break-glass)

6.2 Demo → AWS configuration map

Demo setting	Demo value (do not ship)	AWS target
sqlserver.url	jdbc:sqlserver://sqlserver:1433;databaseName=FhirR4PlatformDemo;encrypt=true;trustServerCertificate=true	RDS endpoint; encrypt=true ; prefer proper trust (not trustServerCertificate=true) once CA known
sqlserver.username/password	sa / PilotFish_Demo1!	Least-privilege SQL login (not sa); from Secrets Manager
fhir.oauth.jwks / issuer	http://keycloak:8080 ... / http://localhost:8112 ...	https://auth.example.com/realms/... or Cognito JWKS/issuer HTTPS URLs
PublicFhirBase (Bulk module)	http://localhost:8110/eip/rest/fhir	https://fhir.example.com/eip/rest/fhir
CapabilityStatement.url	http://localhost:8110/eip/rest/fhir/metadata	Same public HTTPS base + /metadata
OAUTH_* in Web UI	fhir-demo-secret / FhirDemo1!	Secrets; rotate; remove password-grant for prod UIs
FHIR_PUBLIC_BASE_URL / LAN_HINT	192.168.x.x	Public or VPN hostname
Keycloak command	start-dev --import-realm	kc.sh start (prod) + Postgres; hostname-strict HTTPS
is_debug	true	false in prod

6.3 Example ECS task environment (EIP)

```
# Inject via secrets / env – then render environment-settings.conf at entrypoint
SQLSERVER_JDBC_URL=jdbc:sqlserver://fhir-rds...amazonaws.com:1433;
    databaseName=FhirR4PlatformDemo;encrypt=true;loginTimeout=30
SQLSERVER_USER=fhir_app
SQLSERVER_PASSWORD={{resolve:secretsmanager:fhir/rds/app:SecretString:password}}
FHIR_OAUTH_JWKS=https://auth.example.com/realms/fhir/protocol/openid-connect/certs
FHIR_OAUTH_ISSUER=https://auth.example.com/realms/fhir
PUBLIC_FHIR_BASE=https://fhir.example.com/eip/rest/fhir
CATALINA_OPTS=-Xms1024M -Xmx3072M -server -XX:+UseG1GC
JAVA_OPTS=-Djava.security.egd=file:///dev/urandom
```

Recommended pattern: keep a templated environment-settings.conf and have demo-entrypoint.sh (or a thin wrapper) substitute \$\$/env values before Tomcat starts. Avoid rebuilding the image to change JDBC URLs.

7. IAM roles & policies (checklist)

Role	Attach / allow	Purpose
ecsTaskExecutionRole	AmazonECSTaskExecutionRolePolicy + secretsmanager:GetSecretValue on fhir/* + kms:Decrypt if CMK	Pull ECR image; inject secrets; write CW logs
ecsTaskRole-eip	elasticfilesystem:ClientMount/Write on access point; optional s3:PutObject on export bucket; logs	Runtime AWS API calls from EIP (if any)
ecsTaskRole-webui	Minimal; usually none beyond CW logs	Web UI should not need broad AWS power
ci-deploy-role	ecr:PutImage, ecs:UpdateService, iam:PassRole (scoped), s3 sync docs optional	GitHub Actions / CodePipeline deploys
rds-monitoring / enhanced	AWS managed monitoring roles	Ops visibility

Least privilege tip: do not give the EIP task AdministratorAccess “temporarily.” Bulk export today writes local NDJSON under EFS — S3 permissions are only needed if you add a copy/lifecycle job.

8. Data plane: SQL migrations, Bulk files, CapStatement URLs

8.1 Schema

- Apply in order: sql/01_init.sql → 02_phase2_search.sql → 03_phase3_bundle.sql → 04_phase6_bulk.sql.
- Run as a one-shot ECS task / CodeBuild / SSM session against RDS — same as compose's sqlserver-init.
- Create an app login with rights only on FhirR4PlatformDemo objects (not sysadmin).

8.2 Bulk \$export files

- Jobs metadata: dbo.FhirExportJobs; files: /opt/pilotfish/output/bulk-export/{jobId}/.
- With desiredCount > 1, **EFS (or equivalent shared FS) is required** so status/file GETs can hit any task.
- Set Bulk module PublicFhirBase to the ALB HTTPS FHIR base or Content-Location links break for clients.
- Optional hardening: cron copy completed NDJSON to S3 + lifecycle expire EFS after N days.

8.3 CapabilityStatement / metadata host

Update capability-statement.json (or the XSLT that emits it) so url and any absolute links use the public HTTPS base. Clients and Inferno-like suites fingerprint this. Re-run metadata tests after DNS cutover.

9. Image build & CI/CD sketch

```
# Build (on CI runner with access to licensed pilotfish-eip base)
docker build -f pilotfish/Dockerfile -t $ECR/fhir-eip:$GIT_SHA .
docker build -t $ECR/fhir-webui:$GIT_SHA ./webui
docker push $ECR/fhir-eip:$GIT_SHA && docker push $ECR/fhir-webui:$GIT_SHA

# Deploy
aws ecs update-service --cluster fhir --service eip \
  --force-new-deployment # task def already pinned to :$GIT_SHA

# Post-deploy acceptance (from CI runner in VPC or via public ALB)
export FHIR_BASE=https://fhir.example.com/eip/rest/fhir
export WEBUI=https://fhir-ops.example.com # if exposed
python3 tools/run_interface_tests.py --wait
```

- Pin digests in task definitions; avoid floating :latest in prod.
- Scan images (ECR enhanced scanning / Trivy) — custom JARs + Tomcat surface matter.
- Separate AWS accounts or at least separate clusters for sandbox vs prod.

10. Swapping Keycloak for Amazon Cognito (optional)

The EIP JWT processor needs a reachable JWKS and tokens whose iss matches configured issuer. Cognito satisfies that without running Keycloak:

- Create User Pool + App client (client_credentials and/or auth code+PKCE).
- Issuer: `https://cognito-idp.{region}.amazonaws.com/{userPoolId}`
- JWKS: `{issuer}/.well-known/jwks.json`
- Point `fhir.oauth.*` at those URLs; remove Keycloak ECS service.
- Re-test write + `/export` with Cognito access tokens; update Web UI `OAuth.*` accordingly.
- Scopes/claims: map Cognito groups/custom attributes if you later enforce patient compartments.

Cognito is **not** full SMART App Launch by itself. If the buyer requires SMART EHR launch, plan that as a separate workstream (see Expert Due Diligence PDF).

11. Fast path: single EC2 Docker Compose (staging only)

- EC2 (m6i.xlarge+), Amazon Linux 2023, Docker Engine, 100GB+ gp3.
- Open SG: 443 (ALB or host nginx) only; SSH via SSM Session Manager (no 22/world).
- Clone demo; replace passwords; set PublicFhirBase + OAuth issuer to the EC2/ALB hostname.
- Put nginx/Caddy with Let's Encrypt or ACM via ALB in front of 8110/8111/8112.
- Stop-gap for demos — promote to ECS+RDS before any PHI or external partner traffic.

12. Go-live acceptance checklist

#	Check	Pass criteria
1	TLS	fhir + auth hosts serve valid certs; HTTP→HTTPS redirect
2	metadata	GET /metadata returns CapStatement; url host is public
3	CRUD + search	Living test plan CRUD/search suites green
4	AuthN	Write without Bearer → 401; with valid token → 2xx
5	Validation	Invalid Patient rejected by HAPI validator path
6	Bundle	transaction atomic failure case tested
7	Bulk	kickoff 202 → status 200 → NDJSON downloadable via public URL
8	Secrets	No demo passwords in env, images, or Terraform state plaintext
9	Backups	RDS snapshot restore tested in last 30 days
10	Observability	ALB 5xx + RDS CPU/storage alarms page a human
11	Scale safety	If count≥2, Bulk file fetch works against both tasks
12	Scope honesty	CapStatement deferred items still accurate for this release

13. Rough monthly cost envelope (order-of-magnitude, us-east-1)

Numbers move with commitment discounts and traffic. Use for planning conversations only:

Stack	Ballpark	Includes
Staging (ECS light + RDS SQL Server Express/Web + ALB + NAT)	\$350–900 / mo	Single AZ RDS; low traffic; Keycloak optional
Prod-ish (Multi-AZ RDS + 2 EIP tasks + EFS + WAF + dual NAT)	\$1.5k–4k+ / mo	Before PilotFish license; SQL Server license edition matters a lot
EC2 Compose sandbox	\$80–250 / mo	One box + ALB optional; not HA

14. Demo defaults you must not carry into AWS

Demo default	Risk	Production action
sa / PilotFish_Demo1!	Trivial DB compromise	App login + Secrets Manager + rotation
Keycloak admin/admin + start-dev	Realm takeover	Prod mode, strong admin, no public admin
fhir-demo-secret client secret	Token minting by attackers	New clients; rotate; secret in SM
Open reads (no Bearer)	Data exposure	Decide policy; often require auth on all verbs
trustServerCertificate=true	MITM on JDBC	Install RDS CA; disable trust-all
localhost PublicFhirBase	Broken Bulk clients	Set HTTPS public FHIR base
is_debug=true	Verbose leakage	false
Bind-mount host folders	Lost on task replace	EFS + log drivers
Single-node assumption	Broken Bulk w/ scale-out	Shared FS before desiredCount>1

15. Copy-paste implementation ticket (Epic)

```
Title: Deploy FHIR R4 PilotFish platform to AWS (ECS + RDS)
Context: Lift Clients/Demos/fhir-r4-platform beyond Docker Compose.
Scope: Phase A-C in AWS Deployment Guide; API + Bulk + OIDC; Web UI optional.
Out of scope: US Core IG packaging, full SMART launch, full search grammar,
  Group/Patient $export, CapStatement $validate (see Expert Due Diligence).
Acceptance: $12 checklist green; living test plan against https://fhir...;
  no demo secrets; CapStatementurl uses public host; Bulk NDJSON via HTTPS.
Deps: Licensed pilotfish-eip base in ECR; DNS zone; ACM; VPC.
Artifacts: Terraform/CDK stack, task defs, secrets ARNs, runbook link, test-results.html.
```

16. Related demo artifacts

- DESIGN.md — honest Phase 1–6 scope
- FHIR_R4_Platform_Expert_Due_Diligence.pdf — FHIR viability questions
- FHIR_R4_Platform_Capability_Brief.pdf — stakeholder outcomes
- FHIR_R4_Platform_Test_Plan.pdf + tools/run_interface_tests.py — acceptance automation
- docker-compose.yml — source of truth for local service graph
- pilotfish/demo-eip-root/environment-settings.conf — JDBC + OAuth knobs
- sql/01–04_*.sql — schema migrations

Bottom line: AWS viability for this solution is straightforward as an ECS + RDS + ALB + Secrets + EFS service if you treat OAuth issuer URLs, PublicFhirBase, shared Bulk storage, and secret hygiene as first-class work. It is not a blind Compose lift. Pair this guide with the Expert Due Diligence PDF so infrastructure success is not mistaken for full FHIR-server parity.